

```

import cv2
import numpy as np
import networkx as nx
from shapely.geometry import Polygon, Point, box
from shapely.affinity import rotate, scale
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from itertools import combinations
import random
import math
from copy import deepcopy

#
=====
=====
# 1. GRAFO DE CONCEITOS (BASE DE CONHECIMENTO)
#
=====
=====
class ConceptGraph:
    """
    Grafo acíclico direcionado que armazena conceitos (nós) e relações (arestas).
    Os IDs são strings para facilitar a leitura.
    """
    def __init__(self):
        self.graph = nx.DiGraph()
        # Nós básicos (primitivas)
        self.add_concept("CIRCULO", tipo="primitiva", formula="circulo")
        self.add_concept("RETANGULO", tipo="primitiva", formula="retangulo")
        self.add_concept("TRIANGULO", tipo="primitiva", formula="triangulo")
        # Cores básicas
        self.add_concept("VERMELHO", tipo="cor", rgb=(255,0,0))
        self.add_concept("AZUL", tipo="cor", rgb=(0,0,255))
        self.add_concept("VERDE", tipo="cor", rgb=(0,255,0))
        self.add_concept("AMARELO", tipo="cor", rgb=(255,255,0))
        # Relações comuns
        self.add_edge("CIRCULO", "POSSUI_COR", "VERMELHO")
        self.add_edge("RETANGULO", "POSSUI_COR", "AZUL")
        self.add_edge("TRIANGULO", "POSSUI_COR", "VERDE")
        # Novos conceitos complexos serão adicionados dinamicamente

    def add_concept(self, name, **attrs):
        if name not in self.graph:
            self.graph.add_node(name, **attrs)

    def add_edge(self, from_node, relation, to_node):
        self.graph.add_edge(from_node, to_node, relation=relation)

```

```

def get_concept(self, name):
    return self.graph.nodes.get(name)

def get_relations(self, node):
    return list(self.graph.successors(node))

def add_complex_concept(self, name, formula, base_primitives=None):
    """Cria um conceito composto (ex: PELO_AGRUPADO) com uma fórmula
matemática."""
    self.add_concept(name, tipo="complexo", formula=formula)
    if base_primitives:
        for prim in base_primitives:
            self.add_edge(name, "COMPOSTO_DE", prim)

#
=====
# 2. PROCESSADOR DE IMAGEM (PRÉ-PROCESSAMENTO DETERMINÍSTICO)
#
=====

class ImageProcessor:
    """
    Extrai contornos (via Canny + aproximação poligonal), cores dominantes (K-means)
    e ordena camadas por oclusão (toque de bordas).
    Retorna um grafo (NetworkX) onde cada nó é uma forma com atributos.
    """

    def __init__(self, image, n_colors=5):
        self.image = image
        self.n_colors = n_colors
        self.height, self.width = image.shape[:2]

    def process(self):
        # 1. Detecção de bordas e contornos
        gray = cv2.cvtColor(self.image, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(gray, 50, 150)
        contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        # 2. Extrair formas (polígonos aproximados)
        shapes = []
        for cnt in contours:
            if len(cnt) < 5: # ignora ruído
                continue
            epsilon = 0.02 * cv2.arcLength(cnt, True)
            approx = cv2.approxPolyDP(cnt, epsilon, True)

```

```

if len(approx) >= 3:
    poly = Polygon(approx.reshape(-1, 2))
    if poly.is_valid and poly.area > 50: # área mínima
        shapes.append(poly)

# 3. Extrair cores dominantes via K-means (na imagem inteira)
pixels = self.image.reshape(-1, 3)
kmeans = KMeans(n_clusters=self.n_colors, random_state=0, n_init=10)
labels = kmeans.fit_predict(pixels)
centers = kmeans.cluster_centers_.astype(int)

color_concepts = []
for rgb in centers:
    concept = self._map_color_to_concept(rgb)
    color_concepts.append(concept)

# 4. Construir o grafo com nós e cores
shape_graph = nx.DiGraph()
for i, poly in enumerate(shapes):
    centroid = poly.centroid
    x, y = int(centroid.x), int(centroid.y)
    if 0 <= x < self.width and 0 <= y < self.height:
        pixel_color = self.image[y, x]
        distances = [np.linalg.norm(pixel_color - center) for center in centers]
        closest = np.argmin(distances)
        color = color_concepts[closest]
    else:
        color = "DESCONHECIDO"

    shape_type = self._classify_shape(poly)
    node_id = f"forma_{i}"
    shape_graph.add_node(node_id,
                        tipo=shape_type,
                        polygon=poly,
                        cor=color,
                        area=poly.area,
                        centroide=(centroid.x, centroid.y))
    shape_graph.add_edge(node_id, "POSSUI_COR", color)

# -----
# 5. ORDENAÇÃO POR CAMADA (OCLUSÃO POR TOQUE DE BORDAS)
# -----
nodes = list(shape_graph.nodes(data=True))
n = len(nodes)

# Inicializa layers como 0
for node, data in nodes:
    shape_graph.nodes[node]['layer'] = 0

```

```

# Construir grafo de precedência: aresta A -> B significa "A está na frente de B"
precedence = nx.DiGraph()
for node, _ in nodes:
    precedence.add_node(node)

# Para cada par de polígonos
for i in range(n):
    for j in range(i+1, n):
        node_i, data_i = nodes[i]
        node_j, data_j = nodes[j]
        poly_i = data_i['polygon']
        poly_j = data_j['polygon']

        # Verifica se as bordas (fronteiras) se intersectam
        borders_intersect = poly_i.exterior.intersects(poly_j.exterior)

        # Se houver interseção de bordas
        if borders_intersect:
            ci = poly_i.centroid
            cj = poly_j.centroid
            # Se o centroide de i está dentro de j, então j está na frente de i
            if ci.within(poly_j):
                # j na frente de i => aresta j -> i
                precedence.add_edge(node_j, node_i)
            elif cj.within(poly_i):
                # i na frente de j
                precedence.add_edge(node_i, node_j)
            else:
                # Nenhum centroide dentro do outro: fallback por área (maior atrás)
                if poly_i.area > poly_j.area:
                    # i maior, então j está na frente (menor na frente)
                    precedence.add_edge(node_j, node_i)
                elif poly_j.area > poly_i.area:
                    precedence.add_edge(node_i, node_j)
                # se áreas iguais, mantém ordem arbitrária (não adiciona aresta)
            else:
                # Se não há interseção de bordas, mas há sobreposição de área (um contém o
                outro)
                if poly_i.intersects(poly_j) and not poly_i.touches(poly_j):
                    if poly_i.contains(poly_j.centroid):
                        # j está dentro de i => j na frente de i
                        precedence.add_edge(node_j, node_i)
                    elif poly_j.contains(poly_i.centroid):
                        precedence.add_edge(node_i, node_j)

# Atribuir layers com base na ordenação topológica (se acíclico)
try:

```

```

        # Ordenação topológica: os primeiros são os que estão mais à frente (sem
dependências)
        topo_order = list(nx.topological_sort(precedence))
        # Atribui layer: o primeiro (mais à frente) recebe o maior valor (n-1), o último (mais
atrás) recebe 0
        for idx, node in enumerate(topo_order):
            layer_val = n - 1 - idx
            shape_graph.nodes[node]['layer'] = layer_val
except nx.NetworkXUnfeasible:
    # Se houver ciclo, usa fallback por área
    print("Aviso: ciclo detectado na precedência. Usando ordenação por área.")
    nodes_sorted = sorted(nodes, key=lambda x: x[1]['area'], reverse=True)
    for idx, (node, data) in enumerate(nodes_sorted):
        shape_graph.nodes[node]['layer'] = idx

return shape_graph

def _map_color_to_concept(self, rgb):
    """Mapeia um RGB para um conceito de cor conhecido."""
    colors = {
        "VERMELHO": (255,0,0),
        "AZUL": (0,0,255),
        "VERDE": (0,255,0),
        "AMARELO": (255,255,0),
        "PRETO": (0,0,0),
        "BRANCO": (255,255,255)
    }
    best = None
    best_dist = float('inf')
    for name, rgb_ref in colors.items():
        d = np.linalg.norm(np.array(rgb) - np.array(rgb_ref))
        if d < best_dist:
            best_dist = d
            best = name
    return best

def _classify_shape(self, polygon):
    """Classifica um polígono como círculo, retângulo ou triângulo."""
    coords = list(polygon.exterior.coords)[-1:]
    n = len(coords)
    if n <= 4:
        if n == 3:
            return "TRIANGULO"
        elif n == 4:
            return "RETANGULO"
    else:
        area = polygon.area
        perimeter = polygon.length

```

```

        if perimeter > 0:
            circularity = 4 * math.pi * area / (perimeter * perimeter)
            if circularity > 0.7:
                return "CIRCULO"
    return "DESCONHECIDO"

```

```

#
=====
=====
# 3. GERADOR DE FORMAS (MUTAÇÃO LÓGICA)
#
=====
=====
class ShapeGenerator:
    """
    Gera formas primitivas (círculo, retângulo, triângulo) com parâmetros aleatórios
    e também pode gerar formas complexas a partir de fórmulas do grafo.
    """
    def __init__(self, concept_graph, canvas_size=(400,400)):
        self.graph = concept_graph
        self.canvas_size = canvas_size

    def generate_primitive(self, shape_type, params=None):
        """Gera uma forma primitiva com parâmetros opcionais."""
        if shape_type == "CIRCULO":
            return self._generate_circle(params)
        elif shape_type == "RETANGULO":
            return self._generate_rectangle(params)
        elif shape_type == "TRIANGULO":
            return self._generate_triangle(params)
        else:
            return None

    def _generate_circle(self, params=None):
        if params is None:
            cx = random.randint(50, self.canvas_size[0]-50)
            cy = random.randint(50, self.canvas_size[1]-50)
            r = random.randint(10, 100)
        else:
            cx, cy, r = params.get('cx', 200), params.get('cy', 200), params.get('r', 50)
        pts = []
        for i in range(36):
            angle = 2 * math.pi * i / 36
            x = cx + r * math.cos(angle)
            y = cy + r * math.sin(angle)
            pts.append((x, y))
        return Polygon(pts)

```

```

def _generate_rectangle(self, params=None):
    if params is None:
        x = random.randint(50, self.canvas_size[0]-100)
        y = random.randint(50, self.canvas_size[1]-100)
        w = random.randint(20, 150)
        h = random.randint(20, 150)
        angle = random.uniform(0, 2*math.pi)
    else:
        x, y, w, h = params.get('x', 100), params.get('y', 100), params.get('w', 80),
params.get('h', 60)
        angle = params.get('angle', 0)
    rect = box(x, y, x+w, y+h)
    if angle != 0:
        rect = rotate(rect, angle, origin='center')
    return rect

def _generate_triangle(self, params=None):
    if params is None:
        points = []
        for _ in range(3):
            points.append((random.randint(50, self.canvas_size[0]-50),
random.randint(50, self.canvas_size[1]-50)))
    else:
        points = params.get('points', [(100,100), (200,100), (150,200)])
    return Polygon(points)

def generate_complex(self, concept_name, params=None):
    """Gera uma forma complexa a partir de um conceito do grafo."""
    node = self.graph.get_concept(concept_name)
    if node and node.get('tipo') == 'complexo':
        formula = node.get('formula', "")
        if 'Repetir_Elipse_Distorcida' in formula:
            shapes = []
            for _ in range(30):
                cx = random.randint(0, self.canvas_size[0])
                cy = random.randint(0, self.canvas_size[1])
                rx = random.randint(5, 20)
                ry = random.randint(5, 20)
                angle = random.uniform(0, 2*math.pi)
                ellipse = self._generate_ellipse(cx, cy, rx, ry, angle)
                shapes.append(ellipse)
            return shapes
        else:
            return
    self.generate_primitive(random.choice(["CIRCULO", "RETANGULO", "TRIANGULO"]))
    else:
        return None

```

```

def _generate_ellipse(self, cx, cy, rx, ry, angle):
    """Gera um polígono elíptico."""
    pts = []
    for i in range(36):
        t = 2 * math.pi * i / 36
        x = cx + rx * math.cos(t) * math.cos(angle) - ry * math.sin(t) * math.sin(angle)
        y = cy + rx * math.cos(t) * math.sin(angle) + ry * math.sin(t) * math.cos(angle)
        pts.append((x, y))
    return Polygon(pts)

```

#

```

=====
=====

```

4. COMPARADOR DE GRAFOS (DISTÂNCIA DE EDIÇÃO)

#

```

=====
=====

```

class Grapher:

"""

Calcula a distância de edição entre dois grafos de formas (nós e arestas).

Usa uma heurística baseada em similaridade geométrica e de atributos.

"""

@staticmethod

def compute_distance(graph1, graph2):

"""

Retorna um valor escalar que representa o custo para transformar graph1 em graph2.

Quanto menor, mais similares.

"""

nodes1 = list(graph1.nodes(data=True))

nodes2 = list(graph2.nodes(data=True))

if abs(len(nodes1) - len(nodes2)) > 5:

return float('inf')

total_cost = 0.0

matched2 = set()

for node1_id, data1 in nodes1:

best_cost = float('inf')

best_match = None

for node2_id, data2 in nodes2:

if node2_id in matched2:

continue

cost = Grapher._node_similarity(data1, data2)

if cost < best_cost:

best_cost = cost

best_match = node2_id

if best_match is not None:


```

        total_cost += best_cost
        matched2.add(best_match)
    else:
        total_cost += 1.0

    for node2_id, data2 in nodes2:
        if node2_id not in matched2:
            total_cost += 1.0

    return total_cost

@staticmethod
def _node_similarity(data1, data2):
    """Custo de similaridade entre dois nós (0 = idênticos, >0 = diferentes)."""
    cost = 0.0
    if data1.get('tipo') != data2.get('tipo'):
        cost += 0.5
    if data1.get('cor') != data2.get('cor'):
        cost += 0.5
    area1 = data1.get('area', 0)
    area2 = data2.get('area', 0)
    if area1 > 0 or area2 > 0:
        cost += abs(area1 - area2) / (area1 + area2 + 1e-6) * 0.3
    c1 = data1.get('centroide', (0,0))
    c2 = data2.get('centroide', (0,0))
    dist = np.linalg.norm(np.array(c1) - np.array(c2))
    cost += dist / 400.0 * 0.2
    return cost

```

#

=====

=====

5. MOTOR CRIATIVO (APRENDIZADO E GERAÇÃO)

#

=====

=====

class CreativeEngine:

"""

Orquestra o ciclo de aprendizado: decomposição → geração → comparação → mutação.
Gerencia a criatividade com recompensa por novidade.

"""

```

def __init__(self, concept_graph, canvas_size=(400,400)):
    self.graph = concept_graph
    self.canvas_size = canvas_size
    self.generator = ShapeGenerator(concept_graph, canvas_size)
    self.grapher = Grapher()
    self.history = [] # armazena formas já geradas para calcular novidade

```

```
self.best_grafo = None
self.best_distance = float('inf')
self.stagnation_counter = 0
self.max_stagnation = 10
```

```
def learn_from_image(self, image):
```

```
    """
```

```
    Recebe uma imagem, processa e tenta aprender a estrutura gerando formas
    até estagnação.
```

```
    Retorna o grafo aprendido e o grafo alvo.
```

```
    """
```

```
    # 1. Processar imagem alvo
```

```
    processor = ImageProcessor(image)
```

```
    target_graph = processor.process()
```

```
    print("Grafo alvo extraído com", len(target_graph.nodes), "nós.")
```

```
    # 2. Loop de geração/mutação
```

```
    best_graph = None
```

```
    best_dist = float('inf')
```

```
    no_improve = 0
```

```
    current_graph = self._generate_random_graph()
```

```
    current_dist = self.grapher.compute_distance(current_graph, target_graph)
```

```
    best_graph = current_graph
```

```
    best_dist = current_dist
```

```
    for attempt in range(100):
```

```
        new_graph = self._mutate_graph(current_graph)
```

```
        new_dist = self.grapher.compute_distance(new_graph, target_graph)
```

```
        if new_dist < best_dist:
```

```
            best_graph = new_graph
```

```
            best_dist = new_dist
```

```
            no_improve = 0
```

```
            print(f"Melhoria! Distância: {best_dist:.3f}")
```

```
        else:
```

```
            no_improve += 1
```

```
    current_graph = best_graph
```

```
    current_dist = best_dist
```

```
    if no_improve >= self.max_stagnation:
```

```
        print("Estagnação atingida. Parando.")
```

```
        break
```

```
    self.best_grafo = best_graph
```

```
    self.best_distance = best_dist
```

```
    return best_graph, target_graph
```

```

def _generate_random_graph(self):
    """Gera um grafo com uma forma aleatória (primitiva)."""
    shape_type = random.choice(["CIRCULO", "RETANGULO", "TRIANGULO"])
    poly = self.generator.generate_primitive(shape_type)
    g = nx.DiGraph()
    node_id = "forma_0"
    g.add_node(node_id,
               tipo=shape_type,
               polygon=poly,
               area=poly.area,
               centroide=(poly.centroid.x, poly.centroid.y),
               cor="VERMELHO")
    g.add_edge(node_id, "POSSUI_COR", "VERMELHO")
    return g

def _mutate_graph(self, graph):
    """Aplica mutação a um nó do grafo: altera posição, tamanho, rotação ou cor."""
    new_graph = deepcopy(graph)
    if len(new_graph.nodes) == 0:
        return self._generate_random_graph()

    node_id = random.choice(list(new_graph.nodes))
    data = new_graph.nodes[node_id]
    poly = data.get('polygon')

    mutation_type = random.choice(['posicao', 'tamanho', 'rotacao', 'cor'])
    if mutation_type == 'posicao':
        cx, cy = poly.centroid.x, poly.centroid.y
        dx = random.randint(-30, 30)
        dy = random.randint(-30, 30)
        new_poly = self._translate_polygon(poly, dx, dy)
        new_graph.nodes[node_id]['polygon'] = new_poly
        new_graph.nodes[node_id]['centroide'] = (new_poly.centroid.x, new_poly.centroid.y)
    elif mutation_type == 'tamanho':
        factor = random.uniform(0.8, 1.2)
        new_poly = scale(poly, xfact=factor, yfact=factor, origin='center')
        new_graph.nodes[node_id]['polygon'] = new_poly
        new_graph.nodes[node_id]['area'] = new_poly.area
        new_graph.nodes[node_id]['centroide'] = (new_poly.centroid.x, new_poly.centroid.y)
    elif mutation_type == 'rotacao':
        angle = random.uniform(0, 2*math.pi)
        new_poly = rotate(poly, angle, origin='center')
        new_graph.nodes[node_id]['polygon'] = new_poly
        new_graph.nodes[node_id]['centroide'] = (new_poly.centroid.x, new_poly.centroid.y)
    elif mutation_type == 'cor':
        cores = ["VERMELHO", "AZUL", "VERDE", "AMARELO"]
        new_cor = random.choice(cores)

```

```

        new_graph.remove_edge(node_id, "POSSUI_COR")
        new_graph.add_edge(node_id, "POSSUI_COR", new_cor)
        new_graph.nodes[node_id]['cor'] = new_cor

    return new_graph

def _translate_polygon(self, poly, dx, dy):
    """Desloca um polígono."""
    coords = list(poly.exterior.coords)
    new_coords = [(x+dx, y+dy) for x,y in coords]
    return Polygon(new_coords)

def creative_loop(self, iterations=20):
    """
    Loop criativo: gera formas aleatórias e as avalia por validade e novidade.
    Formas novas e válidas são adicionadas ao grafo de conceitos.
    """
    for i in range(iterations):
        if random.random() < 0.7:
            shape_type = random.choice(["CIRCULO", "RETANGULO", "TRIANGULO"])
            poly = self.generator.generate_primitive(shape_type)
            is_complex = False
        else:
            complex_nodes = [n for n, d in self.graph.graph.nodes(data=True) if d.get('tipo') ==
'complexo']
            if complex_nodes:
                concept = random.choice(complex_nodes)
                poly_or_list = self.generator.generate_complex(concept)
                if isinstance(poly_or_list, list):
                    poly = poly_or_list[0] if poly_or_list else None
                else:
                    poly = poly_or_list
                is_complex = True
            else:
                poly = self.generator.generate_primitive("CIRCULO")
                is_complex = False

        if poly is None or not poly.is_valid or poly.area < 10:
            continue

        valid = poly.is_valid and poly.area > 0
        novelty = self._compute_novelty(poly)
        reward = (1.0 if valid else 0.0) + (novelty if novelty > 0 else 0.0)

        print(f"Iteração {i}: validade={valid}, novidade={novelty:.2f},
recompensa={reward:.2f}")

        if reward > 1.5 and is_complex == False:

```

```

concept_name = f"CONCEITO_{i}"
formula = f"Forma gerada por criatividade: {poly}"
self.graph.add_complex_concept(concept_name, formula,
                               base_primitives=[self._classify_poly(poly)])
print(f"Novo conceito criado: {concept_name}")

```

```

self.history.append(poly)

```

```

def _compute_novelty(self, poly):
    """Calcula a distância média para formas no histórico."""
    if not self.history:
        return 1.0
    c = (poly.centroid.x, poly.centroid.y)
    a = poly.area
    distances = []
    for h in self.history[-50:]:
        hc = (h.centroid.x, h.centroid.y)
        ha = h.area
        dist = np.linalg.norm(np.array(c) - np.array(hc)) + abs(a - ha) / 100.0
        distances.append(dist)
    avg_dist = np.mean(distances) if distances else 0
    return min(avg_dist / 200.0, 1.0)

```

```

def _classify_poly(self, poly):
    coords = list(poly.exterior.coords)[-1]
    n = len(coords)
    if n == 3:
        return "TRIANGULO"
    elif n == 4:
        return "RETANGULO"
    else:
        return "CIRCULO"

```

```

#

```

```

=====
=====

```

```

# 6. RENDERIZAÇÃO (MATPLOTLIB)

```

```

#

```

```

=====
=====

```

```

def render_graph(graph, title="Formas"):
    """Exibe o grafo de formas usando Matplotlib."""
    fig, ax = plt.subplots(figsize=(6,6))
    ax.set_xlim(0, 400)
    ax.set_ylim(0, 400)
    ax.set_aspect('equal')
    ax.set_title(title)

```

```

for node, data in graph.nodes(data=True):
    poly = data.get('polygon')
    if poly is None:
        continue
    x, y = poly.exterior.xy
    ax.fill(x, y, alpha=0.5, edgecolor='black', linewidth=2, label=data.get('tipo'))

handles, labels = ax.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax.legend(by_label.values(), by_label.keys())
plt.show()

```

```

#
=====
=====
# 7. EXEMPLO DE USO
#
=====
=====
if __name__ == "__main__":
    # Criar uma imagem de exemplo (um círculo vermelho e um retângulo azul)
    img = np.ones((400, 400, 3), dtype=np.uint8) * 255
    cv2.circle(img, (150, 200), 60, (0, 0, 255), -1) # vermelho
    cv2.rectangle(img, (250, 150), (350, 250), (255, 0, 0), -1) # azul
    cv2.imwrite("exemplo.png", img) # salva para visualização

    # Inicializar grafo conceitual e motor
    concept_graph = ConceptGraph()
    engine = CreativeEngine(concept_graph, canvas_size=(400,400))

    # 1. Aprendizado a partir da imagem
    print("--- APRENDIZADO ---")
    learned_graph, target_graph = engine.learn_from_image(img)

    # 2. Renderizar o grafo alvo e o aprendido
    print("Grafo alvo:")
    render_graph(target_graph, "Grafo Alvo")
    print("Grafo aprendido:")
    render_graph(learned_graph, "Grafo Aprendido")

    # 3. Loop criativo
    print("\n--- LOOP CRIATIVO ---")
    engine.creative_loop(iterations=10)

    # 4. Exibir o grafo de conceitos atualizado
    print("\nConceitos no grafo:", list(concept_graph.graph.nodes))

```